

Assignment 4: Course Correction

due Monday 07/20 at 9:00am (EST)

After finishing this assignment, submit the `assignment-4.py`, `assignment-4.ipynb`, and `assignment-4-logs.txt` files on Canvas.

Introduction

Let's recap!

In general, our assignments will consist of questions where AI usage is not allowed and those where AI use is expected. Questions where AI use is expected will be explicitly marked as such; if a question has no such label, assume that AI use is not permitted.

You will usually be expected to submit three files on Canvas for every assignment. These are: a `.py` file, where you will write any required code, a `.txt` file with any AI chatbot conversations related to the assignment and any other outside sources you use, and a `.ipynb` file that answers any written questions associated with the assignment.

For this assignment, as with all other work for this course, you are expected to follow the academic integrity guidelines in the course syllabus and Cornell's [Code of Academic Integrity](#).

General Tips + Guidelines

There are several tips and guidelines that you should follow for this assignment:

- You will want to run the code in `assignment-4.py` from the terminal. You can use VSCode's terminal or your computer's terminal app.
- You should run your code in the virtual environment we created in Lab 1. If you have trouble figuring out how to activate it, try Googling! If you're still stuck, reach out to me – the earlier the better.
- Run your code regularly as you write it. It's much easier to catch errors if you debug early and often! I highly recommend that you run the included tests for each task directly after writing the function they test.
- The assignment is broken down into many tasks and it's easy to skip one by accident. Before submitting, go through each task to make sure you've completed it.
- For this assignment, you shouldn't use, and don't need, any Python constructs not yet covered in class. You should demonstrate your skills with the Python we've learned so far!

Part 0

This assignment will have you working with real course descriptions (from the Carleton College 2025-2026 Course Catalog)!

Before beginning, make sure that you have downloaded the `assignment-4.pdf` instructions, `assignment-4.py`, and `assignment-4.ipynb` files and the zipped `course-data` folder. You should place everything in the `info1100/assignments` folder and unzip the `course-data` folder.

All code for this assignment will be written in `assignment-4.py`.

Task 1: Fill in the `# Author` and `# NetID` comments at the top of the `.py` file.

Task 2: In the `assignment-4.ipynb` file, fill in the blanks that ask for your name, NetID, and which AI chatbots you use. If you want to use multiple chatbots, specify which one is used for which question!

Part 1 (Partially AI)

As usual, the `assignment-4.py` file already has some code in it. The code in the `if __name__ == '__main__':` section loads each of the 20 course descriptions in the `course-data` folder and stores them as strings in a list variable called `courses`.

Before you start writing code, I again recommend you look at the structure of the course descriptions, as you'll once again be asked to extract information from them!

Task 3: Write the `regex_search_course` function. This function takes in two arguments — a regex pattern and the course string — then looks for matches to the regex pattern in the course string. If there is only one match, return it. Otherwise, return `None`.

Task 4: Write the `get_course_info` function. This function takes one argument: the string representing a course. Writing this function will be more complicated and require more design decisions on your part than functions I've asked you to write in the past! Broadly, this method will ask you to use regex to extract pieces of information from the course description and return a *tuple* representing each course. Specifically, you should extract:

- the course code (e.g. "CS 111")
- the course name (e.g. "Introduction to Computer Science")
- the course description
- when the course is offered: a *set* of strings representing each term or `None` if not provided
- the minimum and maximum # of course credits: the same value if only one credit value is given, the minimum and maximum # of credits you can take if a range is

given, `None` if not provided

- the liberal arts requirements the course fulfills: a *set* of strings representing each requirement or `None` if not provided
 - if the set of requirements includes `‘‘No Exploration’’`, it should be removed — if it is the only requirement, you should return `None`
- the course tags: a *set* of strings representing each tag or `None` if not provided
- the course faculty: a *set* of strings representing each faculty member or `None` if not provided

You are allowed to use the AI chatbot of your choice to help you design the regular expressions but you must include all conversations in the `assignment-4-logs.txt` file and you should write the rest of your code independently. You should make use of the `regex_search_course` function. Since this function asks you to do so much, you’ll want to think carefully about how to structure it; I would recommend creating helper functions to help you write neat code.

You can use the `courses` list in the main body of the code to test your function.

Task 5: In the `assignment-4.ipynb` file under the Task 5 header, describe in a few sentences your reasoning for how you structured `get_course_info` in Task 4.

Task 6: In the main body of the code, replace the `# Create course tuples` comment with code that creates a list of course tuples for all courses in the `courses` list.

Task 7: Write the `joint_requirements` function which takes two arguments, each of which is a tuple representing a course, and returns a set of the liberal arts requirements shared by both courses.

Task 8: In the main body of the code, replace the `# Find overlapping requirements` comment with calls to the `joint_requirements` function with at least three pairs of courses. You can include more if you want to test your function more thoroughly!

Task 9: Your code has a class named `Course` already defined. Write the `create_course` function, which takes in one course tuple as an argument and returns a `Course` object which contains all of the information stored in the course tuple as attributes.

Task 10: In the main body of the code, replace the `# Create Course objects` comment with code that creates a list of `Course` objects for all courses in the `courses` list.

Task 11: Write the `is_offered` function which takes in a course object and a term string as arguments and returns `True` if the course is offered during that term and `False` otherwise.

Task 12: In the main body of the code, replace the `# Check term offerings` comment with at least three calls to the `is_offered` function.

Task 13: Consider the differences between the `joint_requirements` and `is_offered` func-

tion, one of which used the course tuples and one of which used the `Course` objects. Was one object easier to work with? Are there upsides and downsides to each? In the `assignment-4.ipynb` file under the Task 13 header, write a few sentences describing your comparison.

Task 14: This is your free spot task for this assignment! Think about some interesting analysis you want to do with the courses — either the tuples or objects! This should be slightly more complicated than the `joint_requirements` and `is_offered` functions; think about something you would find it difficult to write on your own. You could study the content of an individual course or compare courses. In the `assignment-4.ipynb` file under the Task 14 header, describe your proposed analysis and why you think it's interesting.

Task 15: Using the AI chatbot of your choice, implement the idea you proposed above. In the main body of the code, replace the `# Call your free spot function here` comment with a call to your new function.

Task 16: As always, you should continue prompting the model until you understand your generated code on a high level. In the `assignment-4.ipynb` file under the Task 16 header, describe what the generated code does.